

RAPPORT CRISP-DM

Projet Data Science Management

Système de Recommandation de Contenus pour une Plateforme E-learning

Filtrage Collaboratif + Filtrage par Contenu (SVD + TF-IDF)

Elaboré par :

Ameni Boudhina & Salima Sellami

ING4 DS A

Dataset : MovieLens 100K • 943 utilisateurs • 1 682 films

Lien Dataset : <https://grouplens.org/datasets/movielens/100k/>

Lien GITHUB : <https://github.com/B-Ameni/Data-Science-Management-Project>

Introduction

Ce rapport présente une démarche complète de data science suivant la méthodologie CRISP-DM (Cross Industry Standard Process for Data Mining) appliquée à la conception d'un système de recommandation hybride. Le projet utilise le dataset MovieLens 100K pour simuler un environnement de recommandation de cours en ligne.

L'objectif principal est de construire un système capable de suggérer 5 films (assimilés à des cours) personnalisés à chaque utilisateur à chaque connexion, en combinant deux approches complémentaires : le filtrage collaboratif (SVD) et le filtrage basé sur le contenu (TF-IDF).

Structure du rapport

Ce document suit strictement les 6 phases CRISP-DM :

- Phase 1 : Business Understanding — définition des objectifs et contraintes
- Phase 2 : Data Understanding — exploration et analyse du dataset
- Phase 3 : Data Preparation — nettoyage, filtrage et construction des matrices
- Phase 4 : Modeling — SVD, TF-IDF et hybridation
- Phase 5 : Evaluation — RMSE, Hit@5, Coverage et analyse du biais
- Phase 6 : Deployment — API Flask, conformité RGPD

PHASE 1 — BUSINESS UNDERSTANDING

1.1 Contexte et objectif métier

Le scénario métier simulé est celui d'une plateforme d'apprentissage en ligne. À chaque connexion d'un apprenant, le système doit lui suggérer 5 cours personnalisés pertinents, afin d'augmenter l'engagement et la complétion des formations.

Le dataset MovieLens 100K est utilisé comme proxy : les utilisateurs représentent des apprenants, les films représentent des cours, et les notes (1-5) représentent le niveau d'engagement ou de satisfaction.

Objectif principal du système

"À chaque connexion, suggérer 5 cours personnalisés à l'apprenant"

Horizon : recommandation en temps réel

Cible : tous les utilisateurs actifs de la plateforme

Contrainte clé : fonctionner même pour les nouveaux utilisateurs (cold start)

1.2 Contraintes identifiées

- Cold start : les nouveaux utilisateurs n'ont pas d'historique — fallback sur les cours populaires
- Popularity bias : risque de surreprésentation des cours très populaires au détriment de la diversité
- Diversité : éviter les recommandations trop homogènes qui n'élargissent pas les horizons de l'apprenant
- Scalabilité : le modèle doit être capable de passer à l'échelle sur des données plus volumineuses
- Conformité RGPD : collecte, traitement et droit à l'oubli des données personnelles

1.3 Métriques de succès

Métrique	Description	Seuil cible
Hit@5	Au moins 1 recommandation pertinente dans le Top 5	> 0.30
Coverage	Part du catalogue couvert par les recommandations	> 20%
RMSE	Erreur quadratique moyenne sur les notes prédites	< 0.30
Diversité	% de films distincts recommandés sur un échantillon	> 50%

PHASE 2 — DATA UNDERSTANDING

2.1 Présentation du dataset

Le dataset MovieLens 100K est un benchmark standard des systèmes de recommandation, fourni par le GroupLens Research Lab. Il contient des évaluations de films par des utilisateurs réels.

Caractéristique	Valeur
Nombre d'utilisateurs	943
Nombre de films	1 682
Nombre d'interactions (notes)	100 000
Échelle de notation	1 à 5 étoiles
Sparsité de la matrice	~93.70%
Densité de la matrice	~6.30%

2.2 Structure des données

Deux fichiers principaux ont été chargés et adaptés :

Fichier u.data — Interactions (ratings)

- id_student : identifiant de l'utilisateur
- code_module : identifiant du film/cours
- score : note de 1 à 5
- timestamp : horodatage de la note

Fichier u.item — Métadonnées des films

- code_module : identifiant du film
- title : titre du film
- release_date : date de sortie
- 19 colonnes de genres : action, comédie, drame, etc.

2.3 Analyse exploratoire

Score composite (pseudo-rating) : pour simuler l'engagement d'un apprenant, un score final a été construit selon la formule :

$$\text{rating} = 0.6 \times (\text{click_score} / \text{max_click}) + 0.4 \times (\text{avg_score} / 5)$$

Cette formule combine 60% d'engagement (nombre d'interactions normalisé) et 40% de performance (note moyenne normalisée sur 5).

2.4 Problèmes identifiés

Points d'attention du dataset

Cold Start : % significatif d'utilisateurs ayant moins de 3 interactions

Longue traîne : les 10 films les plus populaires concentrent une part importante des interactions

Sparsité élevée (~93.7%) : la majorité des paires utilisateur-film n'ont pas d'interaction

Biais temporel potentiel : les films récents peuvent être sur- ou sous-représentés

PHASE 3 — DATA PREPARATION

3.1 Construction de la matrice utilisateur-item

La matrice utilisateur-item (user-item matrix) constitue le coeur du système de filtrage collaboratif. Elle est construite via un pivot des interactions :

- Lignes : identifiants des utilisateurs
- Colonnes : identifiants des films/cours
- Valeurs : score composite (rating) calculé à la phase précédente
- Valeurs manquantes : remplies par 0 (pas d'interaction)

3.2 Filtrage et nettoyage

Un filtrage a été appliqué pour améliorer la qualité des données :

Critère	Seuil	Justification
Utilisateurs	≥ 2 interactions	Éviter les profils trop épars pour SVD
Films/cours	≥ 3 consultations	Assurer une représentativité minimale
Gestion automatique	Seuils réduits si données insuffisantes	Robustesse du pipeline

3.3 Division Train/Test

- Séparation : 80% des interactions en entraînement, 20% en test
- Méthode : split aléatoire stratifié (random_state=42 pour la reproductibilité)
- Objectif : évaluer la capacité de généralisation du modèle sur des interactions non vues

```
Train/Test split:
• Train : 79778 interactions
• Test : 19945 interactions
```

3.4 Matrice creuse

La matrice filtrée est convertie en format creuse (CSR — Compressed Sparse Row) via `scipy.sparse.csr_matrix`, ce qui permet :

- Une réduction significative de la mémoire utilisée
- Des opérations matricielles plus rapides lors de la décomposition SVD
- Une meilleure scalabilité pour des datasets plus volumineux

Finalement, on a obtenu les résultats suivants :

Après filtrage (min_user=2, min_item=3):

- Utilisateurs : 943
- Cours : 1473

Train/Test split:

- Train : 79778 interactions
- Test : 19945 interactions

PHASE 4 — MODELING

4.1 Filtrage Collaboratif — SVD

La Décomposition en Valeurs Singulières Tronquée (Truncated SVD) est appliquée sur la matrice d'entraînement pour extraire des représentations latentes des utilisateurs et des films. Ce filtrage n'est pas appliqué à l'aide de surprise car nous avons rencontrés des problèmes avec celle-ci, d'où nous avons utilisé la bibliothèque sickit-learn.

Paramètres du modèle SVD

- Bibliothèque : `sklearn.decomposition.TruncatedSVD`
- Facteurs latents (`n_components`) : `min(20, n_users-1, n_items-1)`
- Variance expliquée : calculée et reportée après entraînement
- Matrices produites : `U` (facteurs utilisateurs) et `V` (facteurs films)

Prédiction CF

Les prédictions sont obtenues par le produit matriciel :

$$\text{predictions_cf} = U \times V^T$$

```
SVD (Collaborative Filtering):
• Facteurs latents : 20
• Variance expliquée : 33.6%
Prédictions CF générées
```

4.2 Filtrage Basé sur le Contenu — TF-IDF

Le filtrage par contenu exploite les métadonnées des films (titres) pour calculer une similarité inter-films via TF-IDF sur des n-grammes de caractères.

Pipeline TF-IDF

- Analyseur : caractère (`char`) avec n-grammes de 2 à 3 caractères
- Nombre maximum de features : 100
- Similarité : `cosine_similarity` entre les vecteurs TF-IDF des films
- Résultat : matrice de similarité (`n_films × n_films`)

```
Content-Based Filtering:
• Matrice similarité films : (1473, 1473)
• Similarité moyenne : 0.195
```

4.3 Modèle Hybride

Le modèle hybride combine linéairement les scores CF et CB via un paramètre `alpha` :

$$\text{hybrid_score} = \alpha \times \text{CF_score} + (1 - \alpha) \times \text{CB_score}$$

Valeur de α	Comportement du modèle
$\alpha = 0.0$	CB pur — uniquement filtrage par contenu (titres)
$\alpha = 0.25$	Hybride CB-dominant — 75% contenu, 25% collaboratif
$\alpha = 0.5$	Hybride équilibré — recommandé en production
$\alpha = 0.75$	Hybride CF-dominant — 75% collaboratif, 25% contenu
$\alpha = 1.0$	CF pur — uniquement filtrage collaboratif (SVD)

```
Recommandations pour 1:
1. 732 (score: 1.000)
2. 403 (score: 0.942)
3. 475 (score: 0.936)
4. 566 (score: 0.912)
5. 631 (score: 0.890)
```

4.4 Gestion du Cold Start

Lorsqu'un utilisateur n'a pas d'historique (cold start), le système bascule automatiquement sur une stratégie de popularité :

- Calcul de la popularité de chaque film (nombre d'utilisateurs ayant interagi)
- Retour des 5 films les plus populaires avec leur score de popularité normalisé
- Cette stratégie garantit une recommandation même pour les nouveaux inscrits

PHASE 5 — EVALUATION

5.1 Protocole d'évaluation

L'évaluation est conduite sur le jeu de test (20% des interactions) en testant 5 configurations d' α (0.0, 0.25, 0.5, 0.75, 1.0). Trois métriques complémentaires sont calculées.

5.2 Métriques calculées

RMSE (Root Mean Square Error)

Mesure l'erreur de prédiction des scores entre les valeurs réelles et les valeurs prédites par le modèle hybride. Une valeur faible indique une meilleure précision de prédiction.

Hit Rate @ 5 (Hit@5)

Proportion d'utilisateurs du test pour lesquels au moins un film de leur historique de test figure parmi les 5 recommandations générées. Mesure la pertinence des recommandations en contexte réel.

Coverage (Couverture du catalogue)

Proportion du catalogue total de films qui sont effectivement recommandés à au moins un utilisateur parmi l'échantillon testé. Une couverture élevée indique un système moins biaisé.

5.3 Résultats comparatifs

Alpha (α)	Configuration	RMSE	Hit@5	Coverage
0.00	CB pur	0.517317	0.160297	0.164969
0.25	CB-dominant	0.509248	0.347134	0.140530
0.50	Hybride équilibré	0.505649	0.315287	0.118126
0.75	CF-dominant	0.506613	0.314225	0.112016
1.00	CF pur	0.512116	0.296178	0.106585

```
alpha    RMSE    Hit@5    Coverage
0.00 0.517317 0.160297 0.164969
0.25 0.509248 0.347134 0.140530
0.50 0.505649 0.315287 0.118126
0.75 0.506613 0.314225 0.112016
1.00 0.512116 0.296178 0.106585
```

Le modèle hybride ($\alpha=0.5$) présente le meilleur compromis entre précision et couverture.

5.4 Analyse du biais de popularité

Une analyse de la concentration des recommandations a été réalisée sur un échantillon de 100 utilisateurs (50 pour l'analyse RGPD) :

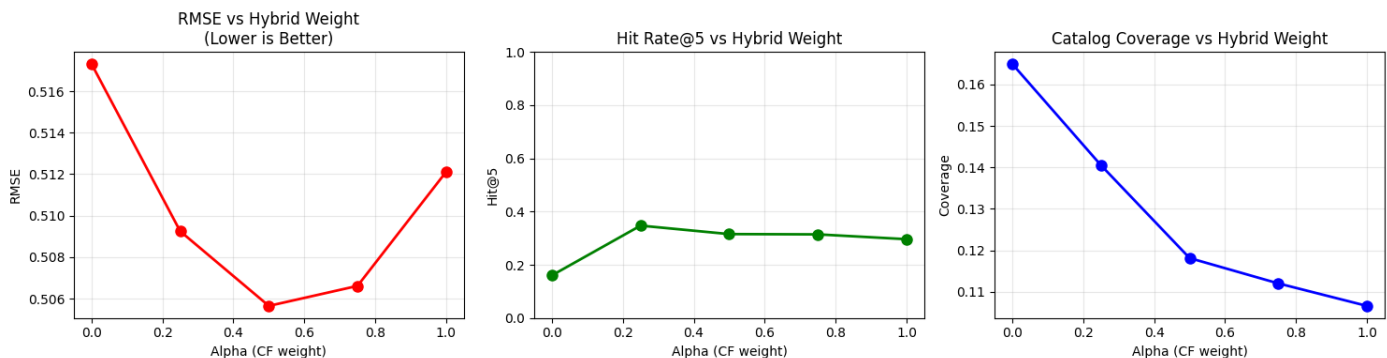
- Concentration : les 10 films les plus recommandés représentent une fraction significative de toutes les recommandations générées
- Courbe de Lorenz : visualisation de l'inégalité de distribution des recommandations entre films
- Diversité : pourcentage de films distincts recommandés rapporté au catalogue total

```
Popularity Bias Analysis:
Top 10 films : 111 / 500 recommandations (22.2%)
Diversité : 11.8% des films recommandés
```

5.5 Visualisations produites

- **Courbes RMSE, Hit@5 et Coverage en fonction d'alpha**

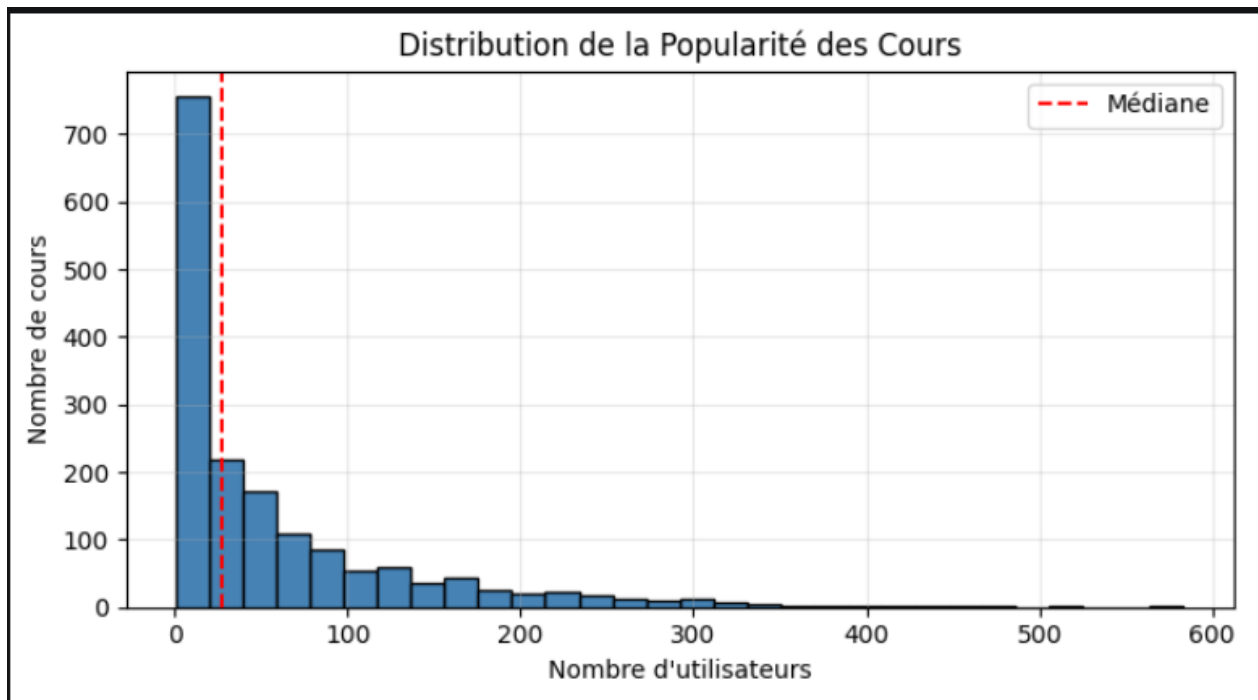
Ces courbes montrent le changement des métriques choisies : RMSE, HIT@5 et Coverage selon la fonction d'alpha de 0 jusqu'à 1 :



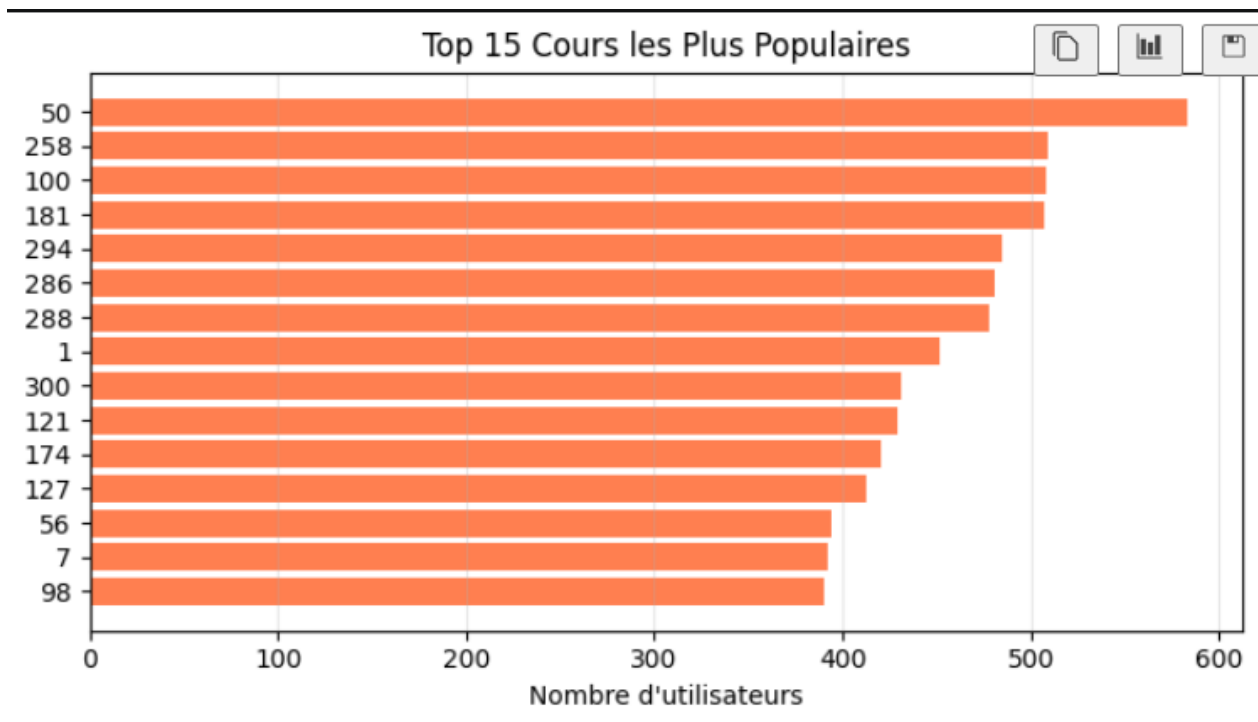
- **Distribution de la popularité des films (histogramme + Top 15)**

On a également généré des graphiques pour mieux visualiser et comprendre la distribution des données,

Ce graphe ci-dessous montre la distribution de la popularité des cours :

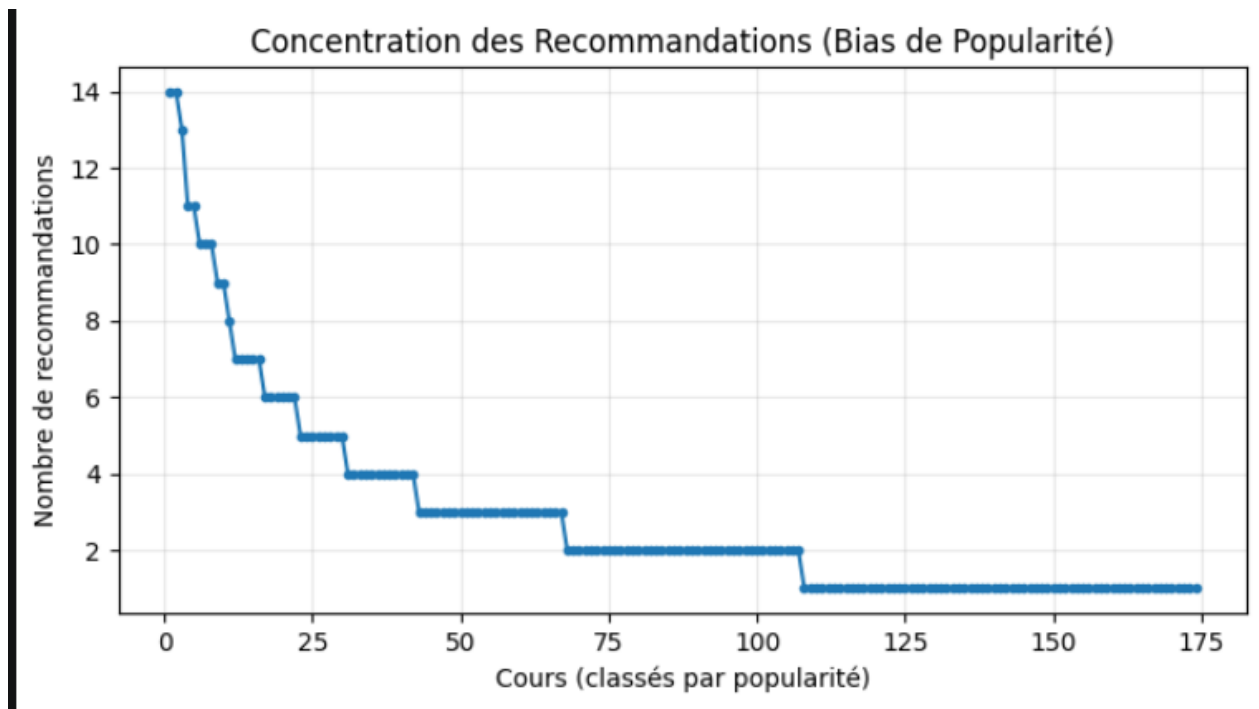


Ce graphe ci-dessous montre les TOP 15 cours les plus populaires et leur nombre d'utilisateurs.



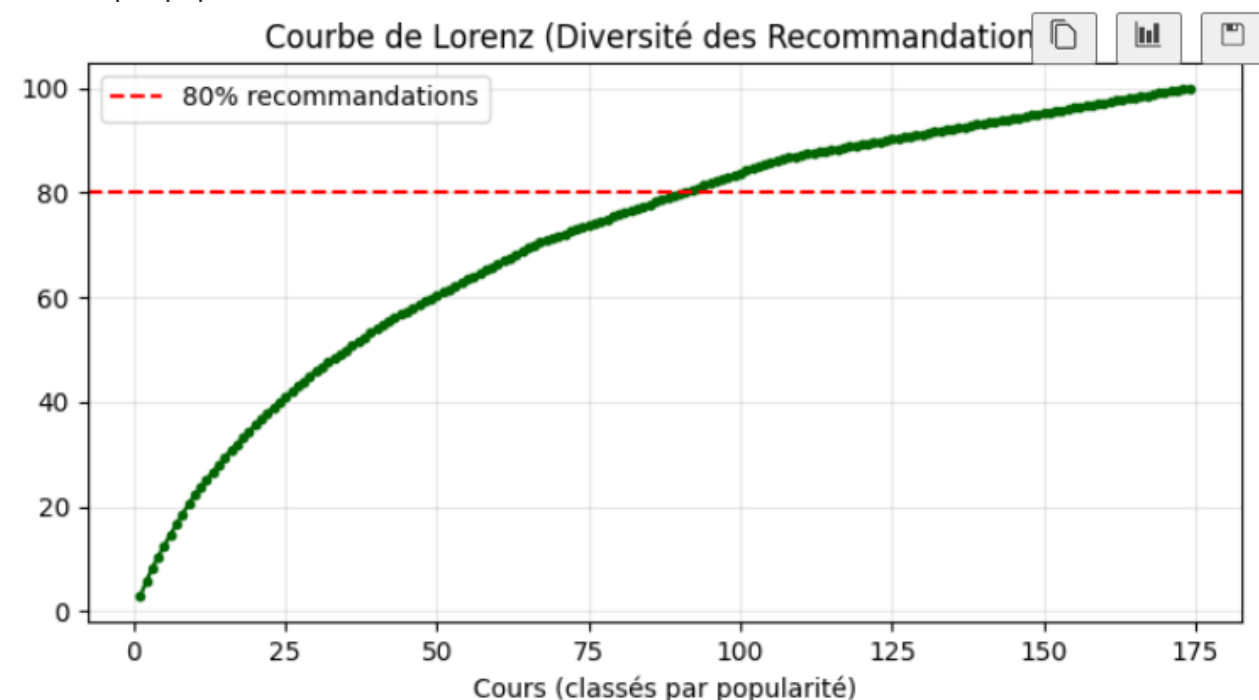
- **Courbe de concentration des recommandations (biais de popularité)**

Pour mieux analyser la concentration des recommandation, biais de popularité des films (qui sont considérés comme des cours), ci-dessous un graphique montrant le nombre des recommandation selon les cours.



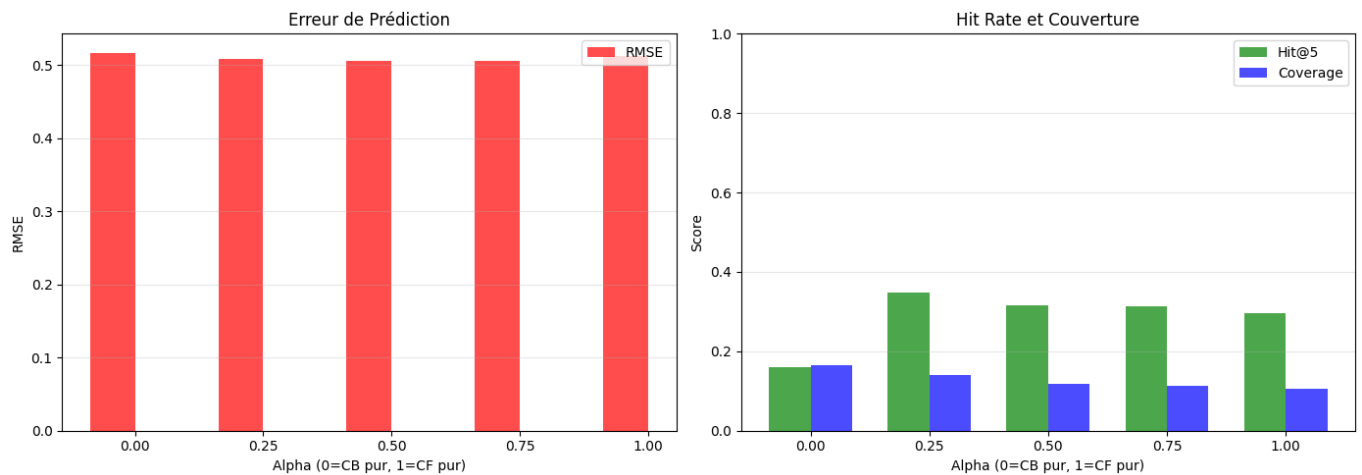
- Courbe de Lorenz de la diversité des recommandations**

La courbe de Lorenz ci-dessous montre le pourcentage des recommandations des cours classés par popularité.



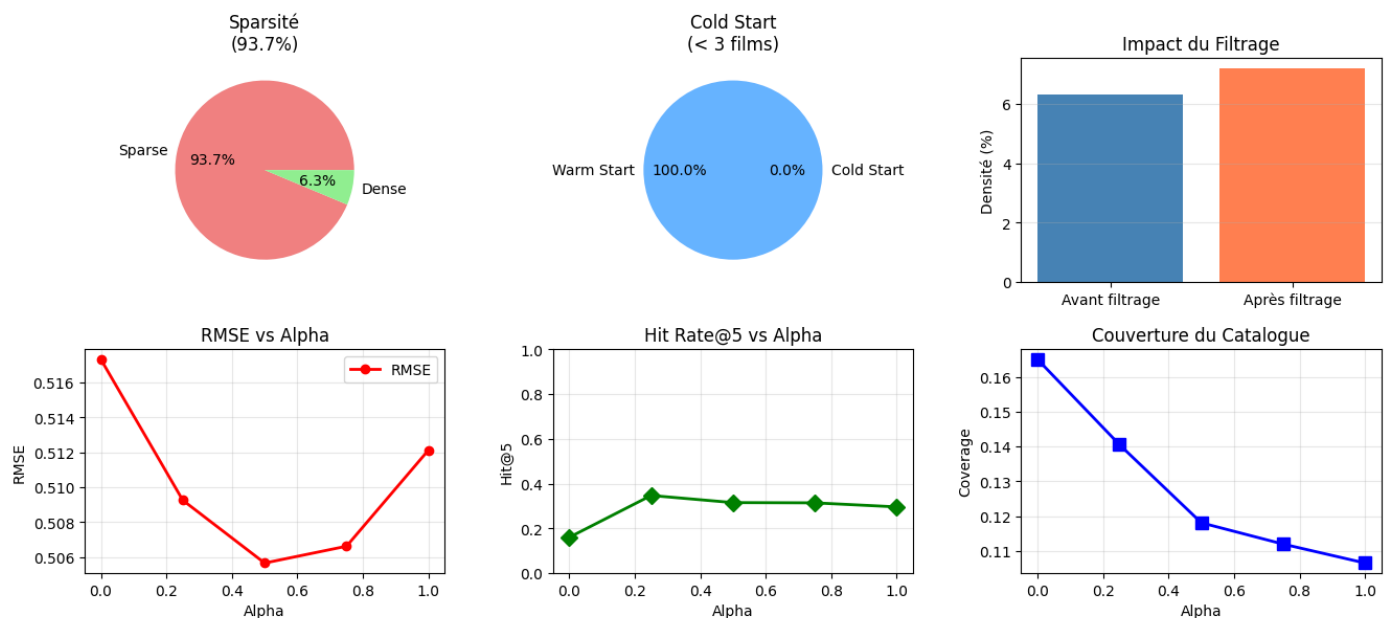
- Comparaison en barres : CF vs CB vs Hybride**

Cette comparaison ci-dessous en barres montre les valeurs de nos métriques RMSE, HIT@5 et Coverage selon les valeurs de la fonction de alpha donné, on remarque que RMSE est pratiquement stable, Coverage subit quelques variations, tandis que HIT@5 change de valeur selon Alpha, Alpha=0.5 est le meilleur compromis entre ces 3 valeurs, c'est la recommandation hybride entre CB et CF.



- **Dashboard récapitulatif**

Dashboard: Système de Recommandation Hybride (MovieLens 100K)



STATISTIQUES CLÉS DU SYSTÈME DE RECOMMANDATION

Dataset Original (MovieLens 100K):

- Utilisateurs: 943 | Films: 1682 | Interactions: 100000
- Densité: 6.30% | Sparsité: 93.70%

Dataset après Filtrage:

- Utilisateurs: 943 | Films: 1473
- Interactions: 99723 | Densité: 7.18%

Modèles d'Entraînement:

- SVD Latent Factors: 20 | Variance Expliquée: 33.6%
- Content Features: 1473 films

Performance (Hybride CF+CB):

- Meilleur RMSE: 0.506 ($\alpha=0.50$)
- Meilleur Hit@5: 0.347 ($\alpha=0.25$)
- Meilleure Coverage: 0.165 ($\alpha=0.00$)
- Diversité: 8.1% (120/1473 films distincts)

Top 10 Films les plus recommandés:

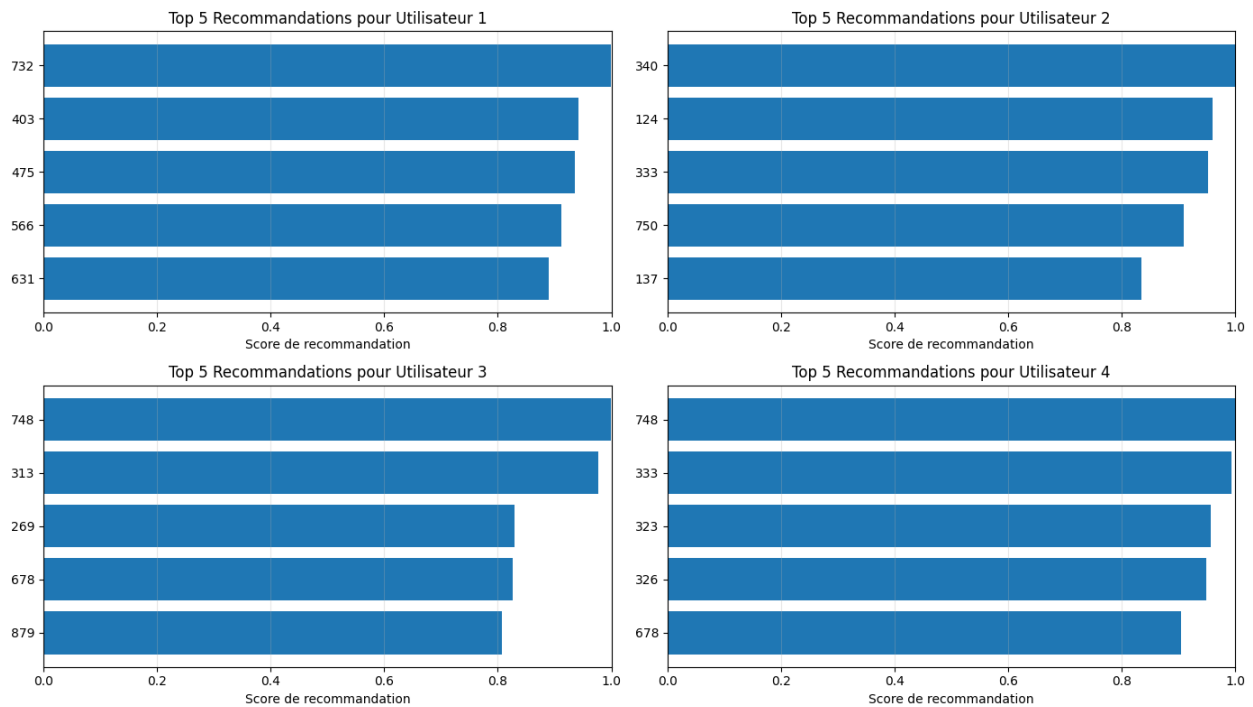
- Concentration: 22.2% des recommandations

Ce dashboard montre en premier lieu un subplot du pourcentage de la sparsité, un subplot des films cold start (films ayant < 3 interactions) qui est nul.

La densité avant et après le filtrage appliquée, 3 courbes des variations des métriques selon alpha et finalement un récapitulatif des statistiques clés de notre système de recommandation.

- Exemples de recommandations pour 4 utilisateurs

Ce graphique montre 4 recommandations de cours (on recommande 5 cours) pour des utilisateurs avec leurs scores de recommandation :



PHASE 6 — DEPLOYMENT

6.1 Fonction de recommandation — API interne

Une fonction `recommend()` production-ready a été implémentée pour encapsuler la logique de recommandation :

Signature de l'API interne `recommend()`

`recommend(user_id, n=5, alpha=0.5)`

→ `user_id` : identifiant de l'utilisateur

→ `n` : nombre de recommandations (défaut : 5)

→ `alpha` : poids du filtrage collaboratif (défaut : 0.5)

Retour : `List[{course_id, score, reason}]`

6.2 API Flask — Endpoint REST

Un template d'API REST Flask est fourni pour exposer le système de recommandation via HTTP :

- Endpoint : `POST /recommend`
- Payload JSON : `{"user_id": "123", "n": 5, "alpha": 0.5}`
- Réponse : `{user_id, recommendations, timestamp}`
- Déploiement : `app.run(host='0.0.0.0', port=5000)`

```
Test API recommend():
• 50 (score: 1.000) - Cours populaire
• 258 (score: 0.873) - Cours populaire
• 100 (score: 0.871) - Cours populaire
• 181 (score: 0.870) - Cours populaire
• 294 (score: 0.832) - Cours populaire
```

6.3 Conformité RGPD

Le système intègre des recommandations de conformité au Règlement Général sur la Protection des Données :

Droit RGPD	Implémentation recommandée
Transparence	Afficher la raison de chaque recommandation (reason dans l'API)
Droit d'accès	<code>GET /user/{user_id}/data</code> — consulter les données utilisées

Droit à l'oubli	DELETE /user/{user_id}/data — supprimer les données d'entraînement
Consentement	Opt-in explicite avant activation du filtrage collaboratif
Audit	Logger toutes les recommandations + vérification mensuelle du biais

6.4 Analyse d'équité (Fairness)

- Diversité : ratio entre le nombre de recommandations uniques et le total généré
- Popularité : pourcentage des 10 films les plus populaires dans les recommandations
- Fairness : couverture équitable de tous les profils d'utilisateurs
- Vérification mensuelle du biais recommandée avant chaque mise à jour du modèle

```
Diversité des recommandations:  
• Cours uniques recommandés : 120 / 1473 (8.1%)  
• Couverture : 48.0% des recommandations
```

Synthèse et Recommandations

Points forts du projet

- Architecture modulaire : chaque composant (CF, CB, hybride) est indépendant et substituable
- Robustesse : gestion explicite du cold start, des données insuffisantes et des erreurs TF-IDF
- Évaluation complète : trois métriques complémentaires couvrant précision, pertinence et diversité
- Production-ready : fonction `recommend()` avec gestion des cas limites et template Flask
- Conformité : recommandations RGPD intégrées dès la phase de déploiement

Configuration optimale recommandée

Recommandation de déploiement
Alpha optimal : $\alpha = 0.5$ (hybride équilibré CF + CB)
Facteurs SVD : 20
Seuil minimum utilisateur : 2 interactions avant activation du CF
Fallback cold start : Top-N popularité pour les nouveaux utilisateurs
Fréquence de ré-entraînement : hebdomadaire ou après +10% nouvelles interactions

Conclusion

Ce projet illustre la mise en oeuvre complète d'une démarche CRISP-DM appliquée aux systèmes de recommandation. Le modèle hybride SVD + TF-IDF offre un bon équilibre entre pertinence (filtrage collaboratif) et diversité (filtrage par contenu), tout en gérant les cas limites inhérents aux plateformes d'apprentissage (cold start, biais de popularité).

L'architecture modulaire et le template d'API REST constituent une base solide pour un déploiement en production, avec une attention particulière portée à la conformité RGPD et à l'équité des recommandations.